Project Report

Tom Barnes

November 4, 2026

1 Project Summary

In this project, I formalised a proof of the fundamental theorem of arithmetic, which says that for any natural number $n > 1$, there exists prime numbers $p_1, p_2, \ldots, p_k$ such that $n = p_1 p_2 \cdots p_k$, and that this set of primes is unique up to reordering, that is if $n = q_1 q_2 \cdots q_l$ for q_j all prime, then $k = l$ and the q_j are a rearrangement of the p_i.

1.1 Proposal Extension

In my original proposal, I stated that I would only be aiming to prove the existence half of the theorem, however, in my first implementation I proved that result and all the necessary important lemmas in under 200 lines. I then refactored the code where possible and cut down the proof to around 110 lines, and I had enough time so decided to prove the uniqueness half as well.

1.2 Definitions

I defined a prime factorisation as a structure `PrimeFactorisation (n : ℕ) (L : List ℕ)` with two fields

- `all_primes` : $\forall p \in$ `L`, p is prime
- `prod_eq` : `L.prod = n`

Apart from this, I used Mathlib's definition of primality: `Nat.Prime n` as stated in the project proposal, as well as `Nat.Coprime m n`.

1.3 Results Used

In the existence half of the proof, I used no major results from Mathlib aside from trivial lemmas relating to arithmetic and list products. For the uniqueness half, for the sake of time, I used a few relevant results from Mathlib, the most important being:

- `Nat.gcdA a b` and `Nat.gcdB a b`
  One of my intermediate lemmas requires Bezout's theorem, which states:

  $$\forall a, b \in \mathbb{N}, \exists\, x, y \in \mathbb{Z}, \text{ s.t. } ax + by = \gcd(a, b)$$

  These results from Mathlib respectively provide the x and y in this identity, and it would have been a significant amount of work to prove this result from scratch.

I also used a few other results relating to primality and coprimality that would have been fairly trivial to prove:

- `Nat.not_prime_iff_exists_dvd_ne` : n is not prime $\iff \exists m,\ m \mid n, m \neq 1$ and $m \neq n$
- `Nat.Prime.coprime_iff_not_dvd` : if $p, n \in \mathbb{N}$ and p is prime, $\gcd(p, n) = 1 \iff p \nmid n$
- `Nat.Prime.dvd_iff_eq` : if $p, n \in \mathbb{N}$, $n > 1$ and p is prime, $n \mid p \iff n = p$
- `Nat.Prime.not_dvd_one` : if p is prime, $p \nmid 1$
- `Nat.Prime.ne_one`, `Nat.Prime.ne_zero` : if p is prime, $p \neq 1$, $p \neq 0$

1.4 Results Proven

Here I state the results I proved and a high-level outline of the proof where it is not trivial.

1.4.1 Existence

1. `prime_factorisation_singleton` : if p is prime, $[p]$ is a factorisation for p

2. `empty_factorisation_is_one` : if a factorisation of n is the empty list then $n = 1$

3. `perm_is_factorisation` : if L_1 is a factorisation of n and L_2 is a permutation of L_1, then L_2 is also a factorisation of n
   Proof: Use `List.Perm.mem_iff` and `List.Perm.prod_eq`.

4. `factorisation_cons` : if $L = [p_1, \ldots, p_k]$ is a factorisation of n and p is prime, then $[p, p_1, \ldots, p_k]$ is a factorisation of $n \cdot p$
   Proof: Use `List.prod_cons` and `List.mem_cons`.

5. `factorisation_append` : if $L_1 = [p_1, \ldots, p_k], L_2 = [q_1, \ldots, q_m]$ are factorisations of n_1 and n_2 respectively, then $[p_1, \ldots, p_k, q_1, \ldots, q_m]$ is a factorisation of $n_1 \cdot n_2$
   Proof: Use induction on the length of L_2. For the base case, use `empty_factorisation_is_one`, and simplify. For the induction step, proceed as follows:

   - show the tail of L_2 is a factorisation for the product of it's elements
   - use the induction hypothesis to combine L_1 with the tail of L_2
   - use `factorisation_cons` to add back the head of L_2
   - show this is a permutation of L_1 combined with L_2 using `List.perm_append_comm_assoc`
   - use `perm_is_factorisation` to conclude

6. `not_prime_exists_factors` : if $n \geq 2$ is composite then it has factors a, b with $1 < a, b < n$
   Proof: Apply `Nat.not_prime_iff_exists_dvd_ne` to obtain a factor m of n that is not equal to 1 or n, then use `grind`, `refine` and `by_cases` tactics to show that m and $\frac{n}{m}$ are both strictly between 1 and n.

7. `exists_factorisation` : every natural number larger than 1 has a prime factorisation
   Proof: Use strong induction on n. In the case n is a prime, apply `prime_factorisation_singleton`. In the other case, apply `not_prime_exists_factors` to obtain nontrivial factors a and b. Apply the induction hypothesis to obtain factorisations for these, then apply `factorisation_append` to finish the proof.

1.4.2 Uniqueness

1. `coprime_dvd_prod` : if $n \mid ab$ and $\gcd(a, n) = 1$ then $n \mid b$
   Proof: Apply Bezout's lemma, multiply by b and show n divides both sides of the resulting expression using Mathlib's `Int.dvd_*` results.

2. `prime_dvd_prod` : if p is prime and $p \mid ab$ then $p \mid a$ or $p \mid b$
   Proof: Trivial if $p \mid a$, otherwise use `Nat.Prime.coprime_iff_not_dvd` to show $gcd(a, p) = 1$ then apply `coprime_dvd_prod`.

3. `prime_dvd_prod_dvd_factor` : if p is prime and $p \mid n$, then p divides a prime factor of n
   Proof: Take a factorisation L of n and use induction on it's length. Use `empty_factorisation_is_one` and `Nat.Prime.not_dvd_one` to show L is nonempty. Then if p divides the product of tail of L, show it is a factorisation of it's product and apply the induction hypothesis. Otherwise $p \mid n$ so apply `prime_dvd_prod` to show p must divide the head of L.

4. `prime_factor_mem_factorisation` : if L_1, L_2 are factorisations for n then $p \in L_1 \implies p \in L_2$
   Proof: Apply `prime_dvd_prod_dvd_factor` to obtain $q \in L_2$ with $p \mid q$, then `Nat.Prime.dvd_iff_eq`.

5. `factorisation_remove_prime` : if L is a factorisation for n and $p \in L$ then removing one copy of p from L is a factorisation for n/p
   Proof: Split L into a head, L_1, singleton containing p and tail L_2, and apply induction. If p is the first element of L_2, take $L = L_2$, otherwise apply the induction hypothesis to split L_2 further. Apply `factorisation_cons` and simplify.

6. `unique_factorisation` : if L_1, L_2 are both factorisations for n then L_2 must be a permutation of L_1
   Proof: Apply induction on L_1. In the case it's empty, show $n = 1$ by `empty_factorisation_is_one` and that L_2 must also be empty. Otherwise, take $p \in L_1$ and apply `prime_factor_mem_factorisation` to get $p \in L_2$. Define L_1' and L_2' as L_1 and L_2 respectively with one copy of p removed. Apply induction on the shorter lists to show they are permutations. Then add back p and show L_1 and L_2 are also permutations.

2 Insights

Since this is an elementary number theory result I have seen before many times, I didn't learn anything new of note regarding this proof, however I learnt a lot about Lean, including:

- **Tactics such as `refine, grind, simpa, obtain`**
- **Proof by induction including strong induction and generalizing on variables**
- **Adhering to Mathlib conventions and Lean coding style**

3 A.I. Usage

For this project I used AI in the following ways:

- **Project structure:** I wrote down a proof of the theorem then used ChatGPT to advise me on which lemmas would be suitable to prove, given the time constraints and my limited knowledge of lean.
- **Mathlib theorem finding and examples:** I frequently used Google AI overview to find the relevant theorem on natural numbers or lists by describing what I wanted to achieve. I also used the AI overview to give me examples of using tactics such as `refine` and `induction generalizing`.